Lecture 3

Accessing Arrays using Pointers

Accessing Arrays using Pointers

Original author of the sides is Jawad Hassan, FAST-NUCES, Islamabad but slides have been modified time by time as per the requirements of the class.

Relationship Between Pointers and Arrays

- Arrays and pointers are closely related
- Arrays are laid out sequentially in memory
- By using the address-of operator (&), we can determine that arrays are laid out

sequentially in memory. For example,

```
int array[5 ] = { 9, 7, 5, 3, 1 };
std::cout << "Element 0 is at address: " << &array[0] <<
'\n'; std::cout << "Element 1 is at address: " << &array[1]
<< '\n';
std::cout << "Element 2 is at address: " << &array[2] << '\n';
std::cout << "Element 3 is at address: " << &array[3] << '\n';
```

- **Each of these memory addresses is 4 bytes apart, which is the size of an integer**

Relationship Between Pointers and Arrays (Cont.)

- Accessing array elements with pointers
  - Assume declarations:

```
int   List[      5      ]; //declare  array
int   *LPtr;                //declare  a pointer
LPtr = List; // Address of first Element of array List
```

Effect:-

-List is an address, no need for &

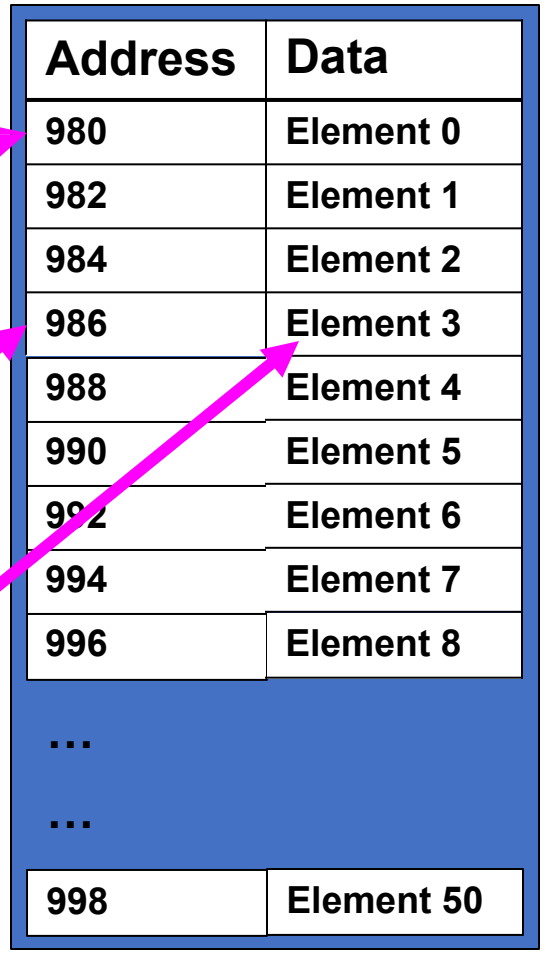
-The LPtr pointer will contain the address of the first element of

array List.

- Element **List[2]** can be accessed by ***(LPtr + 2)**

Accessing 1-Demensional Array Using Pointers

- We know, Array name denotes the memory address of its first slot.
  - Example:
    - **int List [50];**
    - **int *Pointer;**
    - **Pointer = List; //sores address of the first element in Pointer**
- Other slots of the Array (List [50]) can be accessed using by performing Arithmetic operations on Pointer.
- For example the address of (element 4^{th}) can be accessed using:-
  - int *Value = Pointer + 3;
- The value of (element 4^{th}) can be accessed using:-
  - int Value = *(Pointer + 3);



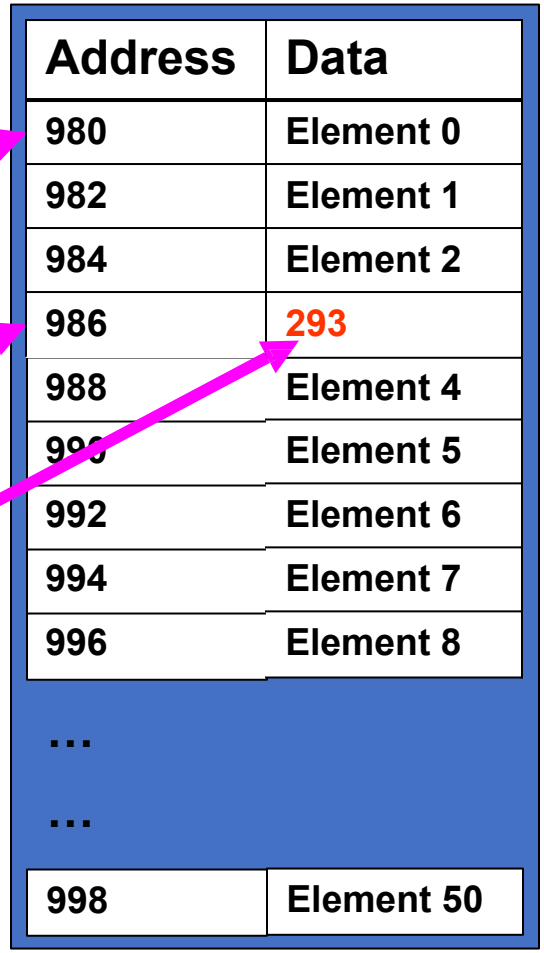
Accessing 1-Demensional Array

```
....
....
      int List [ 50 ];
      int *Pointer;
      Pointer = List;  // Address of first
      Element

      int *ptr;
cout<<*ptr;  //what will it display?
 *ptr is equivalent to List[0]

ptr = Pointer + 3;  // Address of 4th Element

      *ptr = 293;  // 293 value store at 4th element
      address
```

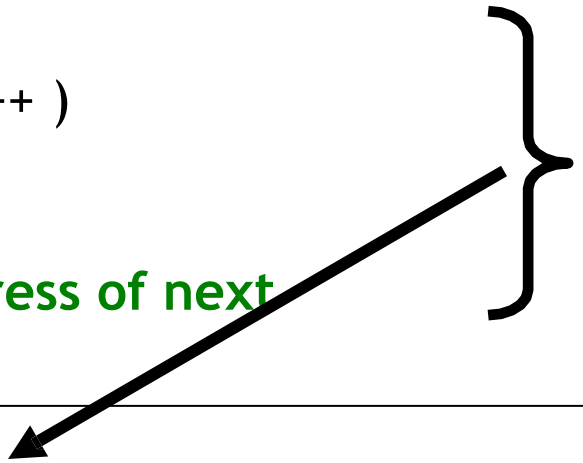


Address	Data
980	Element 0
982	Element 1
984	Element 2
986	293
988	Element 4
990	Element 5
992	Element 6
994	Element 7
996	Element 8
...	
...	
998	Element 50

Accessing 1-Demensional Array

We can access all element of List [50] using Pointers and for loop combinations.

```
...
...
    int List [ 50 ];
    int *Pointer;
    Pointer =
    List;
    for ( int i = 0; i < 50; i++ )
    {
        cout << *Pointer;
        Pointer++; // Address of next
        element
    }
```



This is Equivalent to

```
for ( int loop = 0; loop < 50; loop++ )
    cout <<     Array [ loop ] ;
```

Address	Data
980	Element 0
982	Element 1
984	Element 2
986	Element 3
988	Element 4
990	Element 5
992	Element 6
994	Element 7
996	Element 8
...	
...	
998	Element 50

Question 1

What will be the output?

```
int x = 10;
int *p = &x;

cout << x << endl;
cout << *p << endl;
```

Solution 1

10
10

Question 2

What will be the output?

```
int x = 10;
int *p = &x;

*p = 25;

cout << x << endl;
cout << *p << endl;
```

Solution 2

25
25

Question 3

Consider:
int num = 50;
int *ptr = #

Identify what each expression represents:

num
&num
ptr
*ptr
&ptr

Solution 3

Expression	Meaning
num	Value of num → 50
&num	Address of num
ptr	Address stored in ptr → address of num
*ptr	Value at that address → 50
&ptr	Address of the pointer variable itself

Question 4

Find the error in the following code:

```
int x;
int *p;

*p = 100;

cout << x;
```

Solution 4

The pointer p has not been initialized.

Question 5

What is the output?

```
int x = 10;
int y = 20;

int *p = &x;

*p = y;

cout << x << endl;
cout << y << endl;
```

Solution 5

20
20

Question 6

What will be printed?

```
int x = 50;

int *p1 = &x;
int *p2 = &x;

*p1 = 100;

cout << *p2 << endl;
cout << x << endl;
```

Solution 6

100
100

Question 7

Consider:

```
int x = 10;
int *p = &x;

p = nullptr;
```

What happens to x?
What happens to p?

Solution 7

x remains:10

p becomes: nullptr

Question 8

Consider:

int arr[5] = {10, 20, 30, 40, 50};

What are the values of:

arr[0]
arr[2]
arr[4]

Solution 8

arr[0] = 10
arr[2] = 30
arr[4] = 50

Question 9

What will be the output?

```
int arr[5] = {10, 20, 30, 40, 50};

int *p = arr;

cout << *p << endl;
```

Solution 9

10

Question 10

What will be the output?

```
int arr[5] = {10, 20, 30, 40, 50};

int *p = arr;

cout << *(p + 2);
```

Solution 10

30

Question 11

Consider:

```
int arr[5] = {10, 20, 30, 40, 50};

int *p = arr;
```

Which expression gives the **address of the fourth element**?

Solution 11

A
*(p + 3)
B
p + 3
C
*p + 3
D
p + 4

Correct Answer
B
p + 3

Question 12

What will be the output?

```
int arr[5] = {10, 20, 30, 40, 50};

int *p = arr;

*(p + 2) = 100;

cout << arr[2];
```

Solution 12

100

Question 14

What will be printed?

```
int arr[4] = {10, 20, 30, 40};

int *p = arr;

cout << *p << endl;

p++;

cout << *p << endl;
```

Solution 14

10
20

Question 15

What is the final output?

```
int arr[5] = {10, 20, 30, 40, 50};

int *p = arr;

p++;
p++;

cout << *p;
```

Solution 15

30